22AIE305 INTRODUCTION TO CLOUD COMPUTING



Integrating Monoliths to Cloud-Native Microservices Transformation Intelligence Platform

Team 7 : MonoMelt

Team Members:

- G Prajwal Priyadarshan (CB.SC.U4AIE24214)
- Kabilan K (CB.SC.U4AIE24224)
- Kishore B (CB.SC.U4AIE24227)
- Rahul L S (CB.SC.U4AIE24248)

Abstract

AI powered platform that automates the transformation of legacy monolithic applications into scalable microservice architectures. The system analyzes source code using AST parsing, maps dependencies through a Neo4j graph database, and leverages Large Language Models to generate intelligent decomposition strategies. It provides multiple migration approaches, identifies architectural bottlenecks, and creates a detailed modernization roadmap with cost estimates and implementation phases. To accelerate adoption, MonoMELT also generates production ready Infrastructure as Code, Docker configurations, and microservice boilerplates. By combining graph analytics and AI driven decision making, MonoMELT reduces migration risk and enables faster, more reliable software modernization.

Problem Statement

The Monolithic Bottleneck : As applications grow, legacy monolithic architectures become severely tightly-coupled. This makes them incredibly difficult to scale, slow to deploy, and fragile, as a single bug can crash the entire system.

High-Risk, Manual Migrations : Transitioning from a monolith to distributed microservices manually requires massive engineering effort, takes months of planning, and carries huge operational risk of breaking dependencies or creating a "distributed monolith."

Time-Consuming Infrastructure Configuration : Even when teams successfully map out a new microservice architecture, manually writing the extensive boilerplate code, Dockerfiles, and Kubernetes deployment manifests from scratch for every single service is a highly repetitive, error-prone, and time-consuming process.

Literature Review

S.No	Paper Name	Contribution	Limitation	Relation to MonoMelt	Year
1	Systematic Literature Review of the Cloud-ready Software Architecture (Pozdniakova & Mazeika,)	Defines cloud-ready software principles, cloud-native characteristics and migration requirements	Focuses on cloud readiness rather than automated migration planning.	Provides the architectural foundation for MonoMELT's cloud-native recommendations	2017
2	Software Modernization & Architecture Transformation Journal of Systems and Software– ScienceDirect (S0950584923000848)	Discusses software modernization and architecture transformation techniques	Limited end-to-end automation and decision support.	Supports MonoMELT's automated architecture analysis and modernization workflow	2023
3	Springer Book Chapter – 10.1007/978-981-96-0706-8_12	Presents approaches for modern software architecture and migration toward cloud-native systems.	Emphasizes migration concepts more than practical implementation	Motivates MonoMELT's blueprint generation and migration roadmap	2025
4	Understanding Cloud-Native Applications After 10 Years of Cloud Computing (Kratzke et al.,)	Summarizes cloud-native principles, engineering practices and architectural patterns	Does not address legacy monolith decomposition	Used for cloud-native recommendations and deployment strategy.	2017

S.No	Paper Name	Contribution	Limitation	Relation to MonoMelt	Year
5	Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation (Blinowski et al.,)	Compares scalability, performance and operational trade-offs between monoliths and microservices.	Provides evaluation but not migration assistance.	Supports MonoMELT's performance simulation and cost estimation modules.	2022
6	From Monolith to Microservices: A Classification of Refactoring Approaches (Fritzsch et al.,)	Classifies microservice decomposition and refactoring techniques	Most approaches require manual architectural decisions	Forms the basis of MonoMELT's multi-scenario decomposition engine.	2018
7	Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review (Khan et al.,)	Reviews 35 research papers and introduces the Monolith to Microservices Decomposition Framework (M2MDF), covering decomposition phases, approaches, metrics and datasets	Finds that static, dynamic and evolutionary data are rarely combined and that tool support, standardized metrics and benchmarks remain insufficient.	Directly supports MonoMELT's decomposition engine, dependency analysis and multi-factor service boundary evaluation.	2023

S.No	Paper Name	Contribution	Limitation	Relation to MonoMelt	Year
8	Stepwise Migration of a Monolith to a Microservice Architecture: Performance and Migration Effort Evaluation (Faustino et al.,)	Evaluates stepwise migration using a modular monolith as an intermediate stage and measures migration effort and performance impact.	Supports MonoMELT's phased migration roadmap, performance simulation, rollback planning and modular-monolith fallback recommendation.	Provides the architectural foundation for MonoMELT's cloud-native recommendations	2024
9	A Case Study on Monolith to Microservices Decomposition with Variational Autoencoder-Based Graph Neural Network	Uses static code analysis to create dependency graphs, a VAE to extract component features, and C-means clustering to identify candidate microservices.	Focuses mainly on automated decomposition and clustering rather than cost, security, team ownership, infrastructure or end-to-end migration.	Strongly aligns with MonoMELT's AI-assisted decomposition, graph analysis and multiple candidate service boundaries.	2025
10	Microservice Decomposition via Static and Dynamic Analysis of the Monolith (Krause et al.,)	Combines static analysis with runtime behavior and live trace visualization to identify suitable microservice boundaries in a legacy application.	Requires runtime traces and domain knowledge; focuses primarily on decomposition rather than complete cloud migration planning.	Motivates MonoMELT's dependency graph, data-flow analysis and performance-aware decomposition capabilities.	2020

Research Gap

01 Lack of End-to-End Automated Migration Intelligence

Existing solutions focus only on code decomposition, lacking an end-to-end migration workflow.

02 Limited Context Understanding of Application Architecture

Current methods lack deep business-context understanding of applications.

03 Insufficient Migration Strategy and Decision Support

Limited support for migration decisions, risk analysis, and cost-aware planning.

04 Lack of Cloud-Aware Deployment and Infrastructure Planning

Absence of cloud-native deployment blueprint generation and operational guidance.

Tech Stack

Frontend

- React
- TypeScript
- Tailwind CSS

Api Gateway

- Node.js
- Express.js

AI Services

- Python
- FastAPI
- OpenRouter
- Gemini

AI Sync Worker

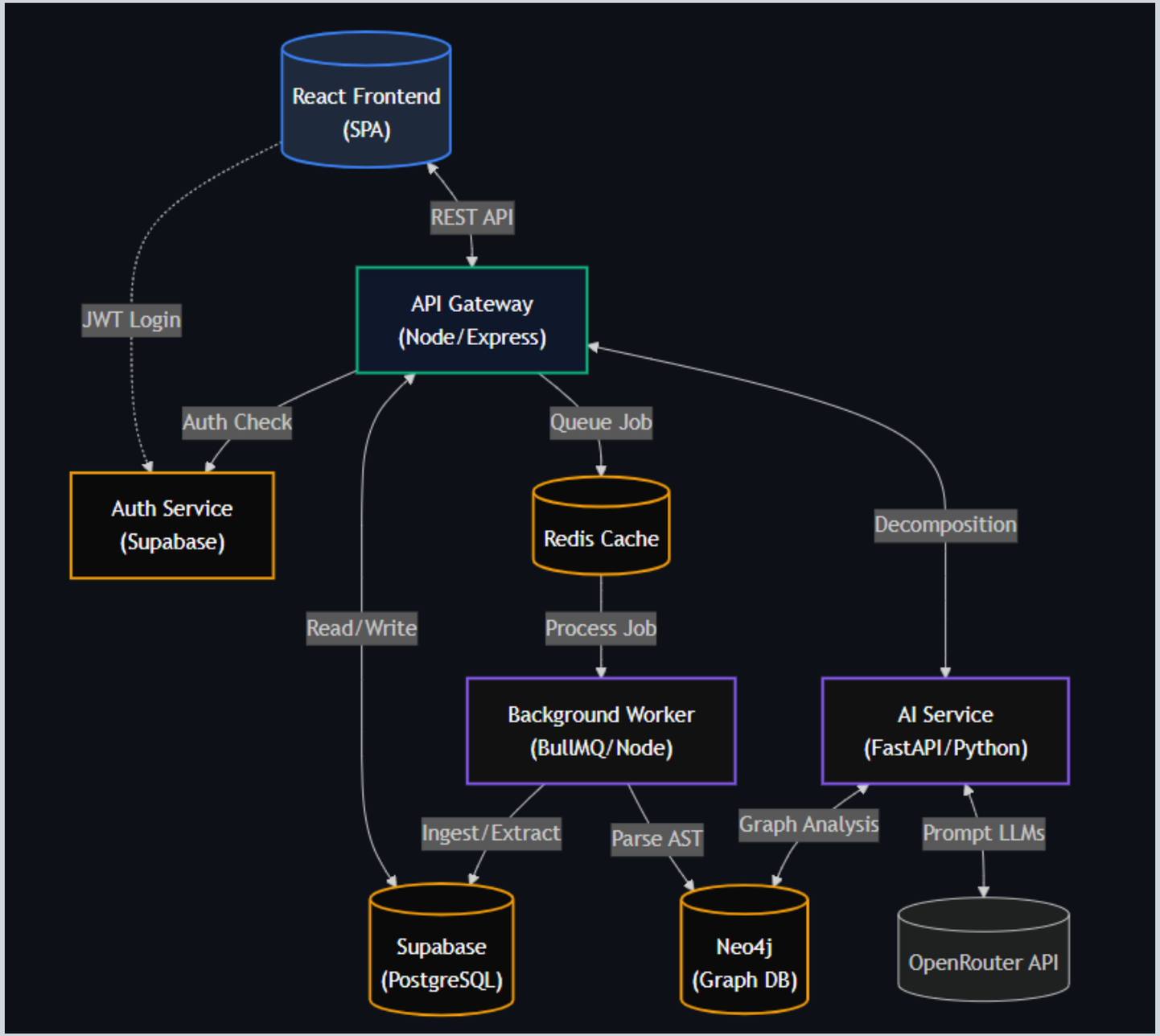
- Redis

Database

- Supabase
- Neo4j

Architecture

- A React SPA frontend authenticates via Supabase and routes requests through a central Node/Express API Gateway.
- For heavy operations like analyzing uploaded codebases, the Gateway offloads tasks via Redis to a BullMQ Background Worker, which parses code structures and stores the dependency graph in Neo4j.
- Finally, a specialized Python/FastAPI AI Service queries this Neo4j graph and interfaces with external LLMs via OpenRouter to generate the intelligent microservice decomposition strategies.



Data Ingestion & Pre-Processing Pipeline

1. Client Upload (React Frontend) :

- The user uploads the legacy monolithic codebase as a .zip archive via the interactive web dashboard.
- The frontend securely transmits the payload with JWT authentication.

2. Request Orchestration (Node.js API Gateway) :

- The Express.js API Gateway receives the file and validates the authentication token via Supabase.
- To prevent the main server from blocking during heavy uploads, it immediately registers the upload in PostgreSQL and pushes a processing job to the queue.

3. Asynchronous Queue (Redis & BullMQ) :

- The Redis cache holds the "Repository Parsing" job in a robust BullMQ queue.
- This ensures the system remains highly responsive and can scale to handle multiple massive codebases simultaneously.

4. Code Extraction & Parsing (Background Worker) :

- A dedicated Node.js Background Worker picks up the job from Redis.
- It unzips the repository into temporary storage and executes raw file analysis (identifying language runtimes, frameworks, and folder structures).
- Crucial Step: It begins building the initial Abstract Syntax Trees (ASTs) to map out classes, functions, and imports.

5. Database Seeding & AI Handoff :

- The parsed structural data is saved into Neo4j (Graph Database) to map dependencies and PostgreSQL for tabular metadata.
- The background worker then signals the Python FastAPI Service, handing off the prepared data to the Multi-Agent AI Framework to begin intelligent decomposition.

Multi-Agent AI Framework for Monolith to Microservices Transformation



- Performs AST-based code parsing
- Extracts modules, classes, functions, APIs, and dependencies
- Builds application dependency graph using Neo4j

Analysis Agent

- Uses LLMs (Gemini/OpenRouter) for semantic understanding
- Detects risks, sensitive data flows, and architectural patterns

Intelligence Agent

- Analyzes coupling and cohesion between components
- Generates 2 decomposition strategies:
  - Conservative
  - Aggressive

Decomposition Agent

- Generates migration sequence
- Estimates cloud cost and performance impact
- Evaluates security and data consistency requirements

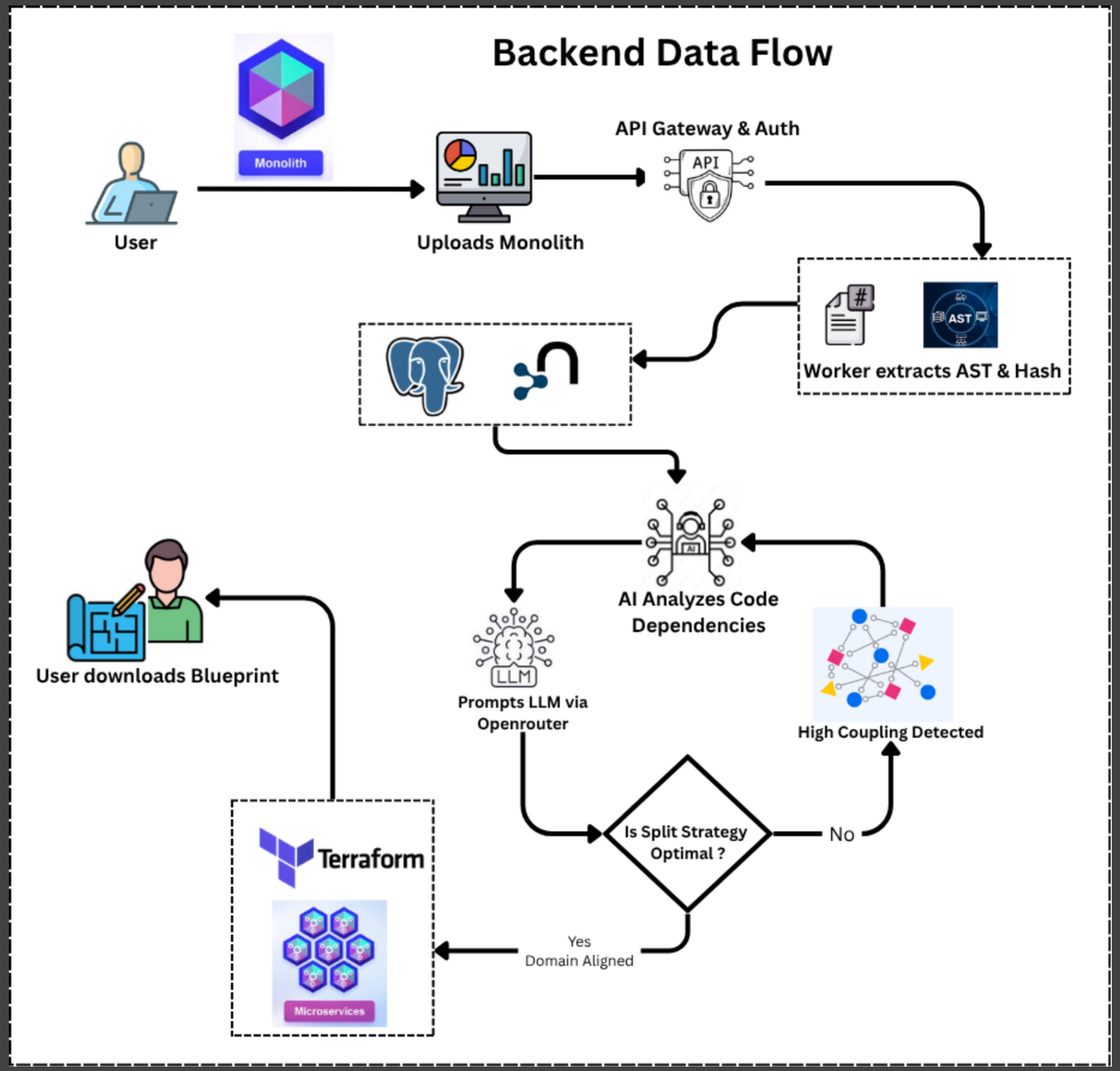
Planning Agent

- Creates deployment specifications
- Generates API contracts and infrastructure templates
- Produces Kubernetes/Docker configurations

Blueprint Agent

An AI-generated **cloud-ready microservices transformation blueprint** containing optimized service architecture, migration roadmap, deployment artifacts, and technical documentation.

Backend Data Flow Diagram



Results & System Outputs

Automated Monolith Decompilation: Successfully built a pipeline that ingests monolithic repositories and uses AI to accurately extract and map out modern microservice boundaries.

Instant Infrastructure-as-Code (IaC): The system successfully generates and exports a production-ready .zip package containing custom Terraform modules, Kubernetes manifests, and Dockerfiles.

Data-Driven Migration Dashboards: Engineered interactive UI dashboards that provide critical architectural metrics, including cost-performance estimates, organizational team mapping, and security compliance matrices.

Comprehensive Output Package: The generated .zip archive provides everything needed to deploy the new architecture, including isolated Dockerfiles and Kubernetes manifests for each microservice, centralized API Gateway routes, OpenAPI contracts, and detailed Architecture Decision Records (ADRs) documenting the extraction strategy.

Github : MonoMELT

Step Wise Outputs : Images

Microservice Zip File : Microservice Zip

Original Repo URL : Original

Enterprise Cloud Architecture (AWS Integration)

1. **Amazon S3 (Simple Storage Service):** Acts as the primary data lake. It stores the massive, raw .zip uploads of monolithic codebases from the user, as well as the unzipped, extracted files ready for processing.

2. **Amazon SQS (Simple Queue Service):** Replaces Redis/BullMQ. When a massive codebase is uploaded to S3, the API Gateway sends a message to SQS. This guarantees that heavy, asynchronous repository parsing jobs are queued and processed reliably without crashing the main server.

3. **AWS Lambda:** Triggered directly by SQS messages. Serverless Lambda functions wake up, pull the codebase from S3, run the heavy Abstract Syntax Tree (AST) parsing to map out the monolith, and then spin back down—saving compute costs.

4. **Amazon EKS (Elastic Kubernetes Service):** The ultimate destination for MonoMelt's output. Because MonoMelt automatically generates Kubernetes deployment manifests (deployment.yaml, service.yaml) for the new microservices, EKS is the exact AWS service where those microservices will run in production.

5. **Amazon ECR (Elastic Container Registry):** MonoMelt writes custom Dockerfiles for every extracted service. ECR is the AWS service used in your CI/CD pipeline to securely store and scan those built Docker container images before they are pushed live to EKS.

6. **Amazon Neptune (Graph Database):** Replaces Neo4j. As Lambda parses the codebase, it stores the complex web of classes, functions, and database dependencies directly into Neptune. This is AWS’s purpose-built graph database, perfect for querying monolithic architectures.

Future Work

- Runtime Traffic Integration: Enhance the static AST analysis by integrating with live APM tools (like Datadog or New Relic) to factor real-time network traffic and database load into the decomposition algorithms.
- Automated PR Generation: Move beyond generating downloadable .zip blueprints by directly integrating with GitHub/GitLab to automatically create Pull Requests for the proposed microservice boundaries.
- Expanded Language Support: Extend the AST parsing capabilities beyond the current supported languages to natively ingest Java, C#, and Go enterprise monoliths.
- Dynamic Cloud Pricing APIs: Connect directly to AWS/GCP/Azure pricing APIs for real-time, dynamic cost projections rather than relying on heuristic estimates.

Conclusion

- Data-Driven Decisions: MonoMELT removes the guesswork from legacy modernization by replacing human intuition with graph algorithms and AI-driven dependency mapping.
- End-to-End Strategic Alignment: We don't just split code; we generate a cohesive strategy that aligns technical boundaries with organizational structures and projects the financial impact.
- Accelerated Modernization: By automating the blueprinting and Infrastructure-as-Code (Terraform) generation, transformation planning is reduced from months of architectural debate to a matter of minutes.
- The Bottom Line: MonoMELT acts as an intelligent co-pilot, ensuring that the leap from a risky monolith to a scalable microservices architecture is safe, cost-effective, and mathematically sound.

Thank You